

esign²—was capable of over 100 petaflops.³ Companies like IBM are now working toward supercomputers capable of exaflops (10^{18} FLOPS).⁴

² <https://en.wikipedia.org/wiki/Folding@home>.

³ <https://en.wikipedia.org/wiki/FLOPS>.

⁴ <https://www.ibm.com/blogs/research/2017/06/supercomputing-weather-model-exascale/>.

The **quantum computers** now under development theoretically could operate at 18,000,000,000,000,000,000 times the speed of today's "conventional computers"!⁵ This number is so extraordinary that in one second, a quantum computer theoretically could do staggeringly more calculations than the total that have been done by all computers since the world's first computer appeared. This almost unimaginable computing power could wreak havoc with blockchain-based cryptocurrencies like Bitcoin. Engineers are already rethinking blockchain to prepare for such massive increases in computing power.⁶

⁵ <https://medium.com/@n.biedrzycki/only-god-can-count-that-fast-the-world-of-quantum-computing-406a0a91fcf4>.

⁶ <https://singularityhub.com/2017/11/05/is-quantum-computing-an-existential-threat-to-blockchain-technology/>.

The history of supercomputing power is that it eventually works its way down from research labs, where extraordinary amounts of money have been spent to achieve those performance numbers, into "reasonably priced" commercial computer systems and even desktop computers, laptops, tablets and smartphones.

Computing power's cost continues to decline, especially with cloud computing. People used to ask the question, "How much computing power do I need on my system to deal with my *peak* processing needs?" Today, that thinking has shifted to "Can I quickly carve out on the cloud what I need *temporarily* for my most demanding computing chores?" You pay for only what you use to accomplish a given task.

Processing the World's Data Requires Lots of Electricity

Data from the world's Internet-connected devices is exploding, and processing that data requires tremendous amounts of energy. According to a recent article, energy use for processing data in 2015 was growing at 20% per year and consuming approximately three to five percent of the world's power. The article says that total data-processing power consumption could reach 20% by 2025.⁷

⁷ <https://www.theguardian.com/environment/2017/dec/11/tsunami-of-ata-could-consume--fifth-global-electricity-by-2025>.

another enormous electricity consumer is the blockchain-based cryptocurrency Bitcoin. Processing just one Bitcoin transaction uses approximately the same amount of energy as powering the average American home for a week! The energy use comes from the process Bitcoin “miners” use to prove that transaction data is valid.⁸

⁸ https://motherboard.vice.com/en_us/article/ywbbpm/bitcoin-mining-electricity-consumption--ethereum-energy-climate-change.

According to some estimates, a year of Bitcoin transactions consumes more energy than many countries.⁹ Together, Bitcoin and Ethereum (another popular blockchain-based platform and cryptocurrency) consume more energy per year than Israel and almost as much as Greece.¹⁰

⁹ <https://digiconomist.net/bitcoin-energy-consumption>.

¹⁰ <https://digiconomist.net/ethereum-energy-consumption>.

Morgan Stanley predicted in 2018 that “the electricity consumption required to create cryptocurrencies this year could actually outpace the firm’s projected global electric vehicle demand—in 2025.”¹ This situation is unsustainable, especially given the huge interest in blockchain-based applications, even beyond the cryptocurrency explosion. The blockchain community is working on fixes.^{2, 3}

¹ <https://www.morganstanley.com/ideas/cryptocurrencies-global-utilities>.

² <https://www.technologyreview.com/s/609480/bitcoin-uses-massive-mounts-of-energy-but-theres-a-plan-to-fix-it/>.

³ <http://mashable.com/2017/12/01/bitcoin-energy/>.

Big-Data Opportunities

The big-data explosion is likely to continue exponentially for years to come. With 50 billion computing devices on the horizon, we can only imagine how many more there will be over the next few decades. It’s crucial for businesses, governments, the military and even individuals to get a handle on all this data.

It’s interesting that some of the best writings about big data, data science, artificial intelligence and more are coming out of distinguished business organizations, such as J.P. Morgan, McKinsey and more. Big data’s appeal to big business is undeniable given the rapidly accelerating accomplishments. Many companies are making significant investments and getting valuable results through technologies in this book, such as big data, machine learning, deep learning and natural-language processing. This is forcing competitors to invest as well, rapidly increasing the need for computing professionals with data-science and computer science experience. This growth is likely to continue for many years.

.7.1 Big Data Analytics

Data analytics is a mature and well-developed academic and professional discipline. The term “data analysis” was coined in 1962,⁴ though people have been analyzing data using statistics for thousands of years going back to the ancient Egyptians.⁵ Big data analytics is a more recent phenomenon—the term “big data” was coined around 2000.⁶

⁴ <https://www.forbes.com/sites/gilpress/2013/05/28/a-very-short-history-of-data-science/>.

⁵ <https://www.flydata.com/blog/a-brief-history-of-data-analysis/>.

⁶ <https://bits.blogs.nytimes.com/2013/02/01/the-origins-of-big-data-n-etymological--detective-story/>.

Consider four of the V’s of big data^{7, 8}:

⁷ <https://www.ibmbigdatahub.com/infographic/four-vs-big-data>.

⁸ There are lots of articles and papers that add many other V-words to this list.

1. Volume—the amount of data the world is producing is growing exponentially.
2. Velocity—the speed at which that data is being produced, the speed at which it moves through organizations and the speed at which data changes are growing quickly.^{9, 10, 11}

⁹ <https://www.zdnet.com/article/volume-velocity-and-variety-nderstanding-the-three-vs-of-big-data/>.

¹⁰ <https://whatis.techtarget.com/definition/3Vs>.

¹¹ <https://www.forbes.com/sites/brentdykes/2017/06/28/big-data-orget-volume-and-variety--focus-on-velocity>.

3. Variety—data used to be alphanumeric (that is, consisting of alphabetic characters, digits, punctuation and some special characters)—today it also includes images, audios, videos and data from an exploding number of Internet of Things sensors in our homes, businesses, vehicles, cities and more.
4. Veracity—the validity of the data—is it complete and accurate? Can we trust that data when making crucial decisions? Is it real?

Most data is now being created digitally in a *variety* of types, in extraordinary *volumes* and moving at astonishing *velocities*. Moore’s Law and related observations have enabled us to store data economically and to process and move it faster—and all at rates growing exponentially over time. Digital data storage has become so vast in capacity, cheap and small

that we can now conveniently and economically retain *all* the digital data we're creating.² That's big data.

² <http://www.lesk.com/mlesk/ksg97/ksg.html>. [The following article pointed us to this Michael Lesk article:

<https://www.forbes.com/sites/gilpress/2013/05/28/a-very-short-history-of-data-science/>.]

The following Richard W. Hamming quote—although from 1962—sets the tone for the rest of this book:

*"The purpose of computing is insight, not numbers."*³

³Hamming, R. W., *Numerical Methods for Scientists and Engineers* (New York, NY., McGraw Hill, 1962). [The following article pointed us to Hamming's book and his quote that we cited: <https://www.forbes.com/sites/gilpress/2013/05/28/a-very-short-history-of-data-science/>.]

Data science is producing new, deeper, subtler and more valuable insights at a remarkable pace. It's truly making a difference. Big data analytics is an integral part of the answer. We address big data infrastructure in [chapter 16](#) with hands-on case studies on NoSQL databases, Hadoop MapReduce programming, Spark, real-time Internet of Things (IoT) stream programming and more.

To get a sense of big data's scope in industry, government and academia, check out the high-resolution graphic.⁴ You can click to zoom for easier readability:

⁴Turck, M., and J. Hao, Great Power, Great Responsibility: The 2018 Big Data & AI Landscape, <http://matrturck.com/bigdata2018/>.

http://matrturck.com/wp-content/uploads/2018/07/Matt_Turck_FirstMark_Big_Data_L

.7.2 Data Science and Big Data Are Making a Difference: Use Cases

The data-science field is growing rapidly because it's producing significant results that are making a difference. We enumerate data-science and big data use cases in the following table. We expect that the use cases and our examples throughout the book will inspire you to pursue new use cases in your career. Big-data analytics has resulted in improved profits, better customer relations, and even sports teams winning more games and championships while spending less on players.^{5, 6, 7}

⁵Sawchik, T., *Big Data Baseball: Math, Miracles, and the End of a 20-Year Losing Streak* (New York, Flat Iron Books, 2015).

⁶Ayres, I., *Super Crunchers* (Bantam Books, 2007), pp. 710.

⁷Lewis, M., *Moneyball: The Art of Winning an Unfair Game* (W. W. Norton & Company, 2004).

data-science use cases		
anomaly detection		
assisting people with disabilities		
auto-insurance risk prediction		
automated closed captioning		
automated image captions		predicting weather-sensitive product sales
automated investing	facial recognition	predictive analytics
autonomous ships	fitness tracking	preventative medicine
brain mapping	fraud detection	preventing disease outbreaks
caller identification	game playing	
cancer diagnosis/treatment	genomics and healthcare	reading sign language
carbon emissions reduction	Geographic Information Systems (GIS)	real-estate valuation
	GPS Systems	recommendation systems
classifying handwriting	health outcome improvement	reducing overbooking
computer vision	hospital readmission reduction	ride sharing
credit scoring	human genome sequencing	risk minimization
crime: predicting locations	identity-theft prevention	robo financial advisors
		security enhancements

crime: predicting recidivism	immunotherapy	self-driving cars
crime: predictive policing	insurance pricing	sentiment analysis
crime: prevention	intelligent assistants	sharing economy
CRISPR gene editing	Internet of Things (IoT) and medical device monitoring	similarity detection
crop-yield improvement	Internet of Things and weather forecasting	smart cities
	inventory control	smart homes
customer churn	language translation	smart meters
customer experience	location-based services	smart thermostats
customer retention	loyalty programs	smart traffic control
customer satisfaction	malware detection	social analytics
customer service	mapping	social graph analysis
customer service agents	marketing	spam detection
customized diets	marketing analytics	spatial data analysis
cybersecurity	music generation	sports recruiting and coaching
	natural-language translation	stock market forecasting
data mining	new pharmaceuticals	student performance assessment
data visualization	opioid abuse prevention	summarizing text
detecting new viruses	personal assistants	telemedicine
diagnosing breast cancer	personalized medicine	terrorist attack prevention
	personalized shopping	theft prevention
diagnosing heart disease	phishing elimination	travel recommendations
	pollution reduction	trend spotting
diagnostic medicine	precision medicine	visual product search
	predicting cancer survival	

disaster-victim identification	predicting disease outbreaks	voice recognition
drones	predicting health outcomes	voice search
dynamic driving routes	predicting student enrollments	weather forecasting
dynamic pricing		
electronic health records		
emotion detection		
energy- consumption reduction		

1.8 CASE STUDY—A BIG-DATA MOBILE APPLICATION

Google’s Waze GPS navigation app, with its 90 million monthly active users,⁸ is one of the most successful big-data apps. Early GPS navigation devices and apps relied on static maps and GPS coordinates to determine the best route to your destination. They could not adjust dynamically to changing traffic situations.

⁸ <https://www.waze.com/brands/drivers/>.

Waze processes massive amounts of **crowdsourced data**—that is, the data that’s continuously supplied by their users and their users’ devices worldwide. They analyze this data as it arrives to determine the best route to get you to your destination in the least amount of time. To accomplish this, Waze relies on your smartphone’s Internet connection. The app automatically sends location updates to their servers (assuming you allow it to). They use that data to dynamically re-route you based on current traffic conditions and to tune their maps. Users report other information, such as roadblocks, construction, obstacles, vehicles in breakdown lanes, police locations, gas prices and more. Waze then alerts other drivers in those locations.

Waze uses many technologies to provide its services. We’re not privy to how Waze is implemented, but we infer below a list of technologies they probably use. You’ll use many of these in chapters 11– 16. For example,

- Most apps created today use at least some open-source software. You'll take advantage of many open-source libraries and tools throughout this book.
- Waze communicates information over the Internet between their servers and their users' mobile devices. Today, such data often is transmitted in JSON (JavaScript Object Notation) format, which we'll introduce in chapter 9 and use in subsequent chapters. The JSON data is typically hidden from you by the libraries you use.
- Waze uses speech synthesis to speak driving directions and alerts to you, and speech recognition to understand your spoken commands. We use IBM Watson's speech-synthesis and speech-recognition capabilities in chapter 13.
- Once Waze converts a spoken natural-language command to text, it must determine the correct action to perform, which requires natural language processing (NLP). We present NLP in chapter 11 and use it in several subsequent chapters.
- Waze displays dynamically updated visualizations such as alerts and maps. Waze also enables you to interact with the maps by moving them or zooming in and out. We create dynamic visualizations with Matplotlib and Seaborn throughout the book, and we display interactive maps with Folium in chapters 12 and 16.
- Waze uses your phone as a streaming Internet of Things (IoT) device. Each phone is a GPS sensor that continuously streams data over the Internet to Waze. In chapter 16, we introduce IoT and work with simulated IoT streaming sensors.
- Waze receives IoT streams from millions of phones at once. It must process, store and analyze that data immediately to update your device's maps, to display and speak relevant alerts and possibly to update your driving directions. This requires massively parallel processing capabilities implemented with clusters of computers in the cloud. In chapter 16, we'll introduce various big-data infrastructure technologies for receiving streaming data, storing that big data in appropriate databases and processing the data with software and hardware that provide massively parallel processing capabilities.
- Waze uses artificial-intelligence capabilities to perform the data-analysis tasks that enable it to predict the best routes based on the information it receives. In chapters 14 and 15 we use machine learning and deep learning, respectively, to analyze massive amounts of data and make predictions based on that data.
- Waze probably stores its routing information in a graph database. Such databases can efficiently calculate shortest routes. We introduce graph databases, such as Neo4J, in chapter 16.
- Many cars are now equipped with devices that enable them to "see" cars and obstacles around them. These are used, for example, to help implement automated braking systems and are a key part of self-driving car technology. Rather than relying on users to report

obstacles and stopped cars on the side of the road, navigation apps could take advantage of cameras and other sensors by using deep-learning computer-vision techniques to analyze images “on the fly” and automatically report those items. We introduce deep learning for computer vision in chapter 15.

1.9 INTRO TO DATA SCIENCE: ARTIFICIAL INTELLIGENCE—AT THE INTERSECTION OF CS AND DATA SCIENCE

When a baby first opens its eyes, does it “see” its parent’s faces? Does it understand any notion of what a face is—or even what a simple shape is? Babies must “learn” the world around them. That’s what artificial intelligence (AI) is doing today. It’s looking at massive amounts of data and learning from it. AI is being used to play games, implement a wide range of computer-vision applications, enable self-driving cars, enable robots to learn to perform new tasks, diagnose medical conditions, translate speech to other languages in near real time, create chatbots that can respond to arbitrary questions using massive databases of knowledge, and much more. Who’d have guessed just a few years ago that artificially intelligent self-driving cars would be allowed on our roads—or even become common? Yet, this is now a highly competitive area. The ultimate goal of all this learning is **artificial general intelligence**—an AI that can perform intelligence tasks as well as humans. This is a scary thought to many people.

Artificial-Intelligence Milestones

Several artificial-intelligence milestones, in particular, captured people’s attention and imagination, made the general public start thinking that AI is real and made businesses think about commercializing AI:

- In a 1997 match between **IBM’s DeepBlue** computer system and chess Grandmaster Gary Kasparov, DeepBlue became the first computer to beat a reigning world chess champion under tournament conditions.⁹ IBM loaded DeepBlue with hundreds of thousands of grandmaster chess games.⁰ DeepBlue was capable of using *brute force* to evaluate up to 200 million moves per second!¹ This is big data at work. IBM received the Carnegie Mellon University Fredkin Prize, which in 1980 offered \$100,000 to the creators of the first computer to beat a world chess champion.²

⁹ https://en.wikipedia.org/wiki/Deep_Blue_versus_Garry_Kasparov.

⁰ [https://en.wikipedia.org/wiki/Deep_Blue_\(chess_computer\)](https://en.wikipedia.org/wiki/Deep_Blue_(chess_computer)).

¹ [https://en.wikipedia.org/wiki/Deep_Blue_\(chess_computer\)](https://en.wikipedia.org/wiki/Deep_Blue_(chess_computer)).

² <https://articles.latimes.com/1997/jul/30/news/mn-17696>.

- In 2011, **IBM’s Watson** beat the two best human Jeopardy! players in a \$1 million match. Watson simultaneously used hundreds of language-analysis techniques to locate

orrect answers in 200 million pages of content (including all of Wikipedia) requiring four terabytes of storage.^{3, 4} Watson was trained with **machine learning** and **reinforcement-learning techniques**.⁵ Chapter 13 discusses IBM Watson and Chapter 14 discusses machine-learning.

³ <https://www.techrepublic.com/article/ibm-watson-the-inside-story-of-how-the-jeopardy--winning-supercomputer-was-born-and-what-it-wants-to-do-next/>.

⁴ [https://en.wikipedia.org/wiki/Watson_\(computer\)](https://en.wikipedia.org/wiki/Watson_(computer)).

⁵ <https://www.aaai.org/Magazine/Watson/watson.php>, *AI Magazine*, Fall 2010.

- Go—a board game created in China thousands of years ago⁶—is widely considered to be one of the most complex games ever invented with 10^{170} possible board configurations.⁷ To give you a sense of how large a number that is, it's believed that there are (only) between 10^{78} and 10^{87} atoms in the known universe!^{8, 9} In 2015, **AlphaGo**—created by Google's DeepMind group—used *deep learning with two neural networks to beat the European Go champion Fan Hui*. Go is considered to be a far more complex game than chess. Chapter 15 discusses neural networks and deep learning.

⁶ <http://www.usgo.org/brief-history-go>.

⁷ <https://www.pbs.org/newshour/science/google-artificial-intelligence-beats-champion--at-worlds-most-complicated-board-game>.

⁸ <https://www.universetoday.com/36302/atoms-in-the-universe/>.

⁹ https://en.wikipedia.org/wiki/Observable_universe#Matter_content.

- More recently, Google generalized its AlphaGo AI to create **AlphaZero**—a game-playing AI that *teaches itself to play other games*. In December 2017, AlphaZero learned the rules of and taught itself to play chess in less than four hours using reinforcement learning. It then beat the world champion chess program, Stockfish 8, in a 100-game match—winning or drawing every game. After *training itself* in Go for just eight hours, AlphaZero was able to play Go vs. its AlphaGo predecessor, winning 60 of 100 games.⁰

⁰ <https://www.theguardian.com/technology/2017/dec/07/alphazero-google-deepmind-ai-beats-champion-program-teaching-itself-to-play-four-hours>.

A Personal Anecdote

When one of the authors, Harvey Deitel, was an undergraduate student at MIT in the mid-

960s, he took a graduate-level artificial-intelligence course with Marvin Minsky (to whom this book is dedicated), one of the founders of artificial intelligence (AI). Harvey:

Professor Minsky required a major term project. He told us to think about what intelligence is and to make a computer do something intelligent. Our grade in the course would be almost solely dependent on the project. No pressure!

I researched the standardized IQ tests that schools administer to help evaluate their students' intelligence capabilities. Being a mathematician at heart, I decided to tackle the popular IQ-test problem of predicting the next number in a sequence of numbers of arbitrary length and complexity. I used interactive Lisp running on an early Digital Equipment Corporation PDP-1 and was able to get my sequence predictor running on some pretty complex stuff, handling challenges well beyond what I recalled seeing on IQ tests. Lisp's ability to manipulate arbitrarily long lists recursively was exactly what I needed to meet the project's requirements. Python offers recursion and generalized list processing (chapter 5).

I tried the sequence predictor on many of my MIT classmates. They would make up number sequences and type them into my predictor. The PDP-1 would "think" for a while—often a long while—and almost always came up with the right answer.

Then I hit a snag. One of my classmates typed in the sequence 14, 23, 34 and 42. My predictor went to work on it, and the PDP-1 chugged away for a long time, failing to predict the next number. I couldn't get it either. My classmate told me to think about it overnight, and he'd reveal the answer the next day, claiming that it was a simple sequence. My efforts were to no avail.

The following day he told me the next number was 57, but I didn't understand why. So he told me to think about it overnight again, and the following day he said the next number was 125. That didn't help a bit—I was stumped. He said that the sequence was the numbers of the two-way crosstown streets of Manhattan. I cried, "foul," but he said it met my criterion of predicting the next number in a numerical sequence. My world view was mathematics—his was broader.

Over the years, I've tried that sequence on friends, relatives and professional colleagues. A few who spent time in Manhattan got it right. My sequence predictor needed a lot more than just mathematical knowledge to handle problems like this, requiring (a possibly vast) world knowledge.

Watson and Big Data Open New Possibilities

When Paul and I started working on this Python book, we were immediately drawn to IBM's Watson using big data and artificial-intelligence techniques like natural language processing (NLP) and machine learning to beat two of the world's best human Jeopardy! players. We realized that Watson could probably handle problems like the sequence predictor because it was loaded with the world's street maps and a whole lot more. That

het our appetite for digging in deep on big data and today’s artificial-intelligence technologies, and helped shape chapters 11– 6 of this book.

It’s notable that all of the data-science implementation case studies in chapters 11– 6 either are rooted in artificial intelligence technologies or discuss the big data hardware and software infrastructure that enables computer scientists and data scientists to implement leading-edge AI-based solutions effectively.

AI: A Field with Problems But No Solutions

For many decades, AI has been viewed as a field with problems but *no* solutions. That’s because once a particular problem is solved people say, “Well, that’s not intelligence, it’s just a computer program that tells the computer exactly what to do.” However, with machine learning (chapter 14) and deep learning (chapter 15) we’re not pre-programming solutions to *specific* problems. Instead, we’re letting our computers solve problems by learning from data—and, typically, lots of it.

Many of the most interesting and challenging problems are being pursued with deep learning. Google alone has thousands of deep-learning projects underway and that number is growing quickly.^{1–2} As you work through this book, we’ll introduce you to many edge-of-the-practice artificial intelligence, big data and cloud technologies.

¹ <http://theweek.com/speedreads/654463/google-more-than-1000-artificial-intelligence-projects-works>.

² <https://www.zdnet.com/article/google-says-exponential-growth-of-ai-is-changing-nature-of-compute/>.

1.10 WRAP-UP

In this chapter, we introduced terminology and concepts that lay the groundwork for the Python programming you’ll learn in chapters 2– 6 and the big-data, artificial-intelligence and cloud-based case studies we present in chapters 11– 6.

We reviewed object-oriented programming concepts and discussed why Python has become so popular. We introduced the Python Standard Library and various data-science libraries that help you avoid “reinventing the wheel.” In subsequent chapters, you’ll use these libraries to create software objects that you’ll interact with to perform significant tasks with modest numbers of instructions.

You worked through three test-drives showing how to execute Python code with the IPython interpreter and Jupyter Notebooks. We introduced the Cloud and the Internet of Things (IoT), laying the groundwork for the contemporary applications you’ll develop in chapters 1– 6.

We discussed just how big “big data” is and how quickly it’s getting even bigger, and

resented a big-data case study on the Waze mobile navigation app, which uses many current technologies to provide dynamic driving directions that get you to your destination as quickly and as safely as possible. We mentioned where in this book you'll use many of those technologies. The chapter closed with our first Intro to Data Science section in which we discussed a key intersection between computer science and data science—artificial intelligence.

2. Introduction to Python Programming

Objectives

In this chapter, you'll:

- Continue using IPython interactive mode to enter code snippets and see their results immediately.
- Write simple Python statements and scripts.
- Create variables to store data for later use.
- Become familiar with built-in data types.
- Use arithmetic operators and comparison operators, and understand their precedence.
- Use single-, double- and triple-quoted strings.
- Use built-in function `print` to display text.
- Use built-in function `input` to prompt the user to enter data at the keyboard and get that data for use in the program.
- Convert text to integer values with built-in function `int`.
- Use comparison operators and the `if` statement to decide whether to execute a statement or group of statements.
- Learn about objects and Python's dynamic typing.
- Use built in function `type` to get an object's type

Outline

.1 Introduction

.2 Variables and Assignment Statements

.3 Arithmetic

.4 Function `print` and an Intro to Single- and Double-Quoted Strings

.5 Triple-Quoted Strings

.6 Getting Input from the User

.7 Decision Making: The `if` Statement and Comparison Operators

.8 Objects and Dynamic Typing

.9 Intro to Data Science: Basic Descriptive Statistics

.10 Wrap-Up

2.1 INTRODUCTION

In this chapter, we introduce Python programming and present examples illustrating key language features. We assume you've read the IPython Test-Drive in [chapter 1](#), which introduced the IPython interpreter and used it to evaluate simple arithmetic expressions.

2.2 VARIABLES AND ASSIGNMENT STATEMENTS

You've used IPython's interactive mode as a calculator with expressions such as

```
In [1]: 45 + 72
Out[1]: 117
```

Let's create a variable named `x` that stores the integer 7:

```
In [2]: x = 7
```

Snippet [2] is a **statement**. Each statement specifies a task to perform. The preceding statement creates `x` and uses the **assignment symbol (=)** to give `x` a value. Most

statements stop at the end of the line, though it's possible for statements to span more than one line. The following statement creates the variable `y` and assigns to it the value 3:

```
In [3]: y = 3
```

You can now use the values of `x` and `y` in expressions:

```
In [4]: x + y
Out[4]: 10
```

Calculations in Assignment Statements

The following statement adds the values of variables `x` and `y` and assigns the result to the variable `total`, which we then display:

```
In [5]: total = x + y

In [6]: total
Out[6]: 10
```

The `=` symbol is not an operator. The right side of the `=` symbol always executes first, then the result is assigned to the variable on the symbol's left side.

Python Style

The *Style Guide for Python Code* ¹ helps you write code that conforms to Python's coding conventions. The style guide recommends inserting one space on each side of the assignment symbol `=` and binary operators like `+` to make programs more readable.

¹ <https://www.python.org/dev/peps/pep-0008/>.

Variable Names

A variable name, such as `x`, is an **identifier**. Each identifier may consist of letters, digits and underscores (`_`) but may not begin with a digit. Python is *case sensitive*, so `number` and `Number` are *different* identifiers because one begins with a lowercase letter and the other begins with an uppercase letter.

Types

Each value in Python has a **type** that indicates the kind of data the value represents. You can view a value's type with Python's built-in **type function**, as in:

```
In [7]: type(x)
Out[7]: int

In [8]: type(10.5)
Out[8]: float
```

The variable `x` contains the integer value 7 (from snippet [2]), so Python displays `int` (short for integer). The value `10.5` is a floating-point number, so Python displays `float`.

2.3 ARITHMETIC

The following table summarizes the **arithmetic operators**, which include some symbols not used in algebra.

Python operation	Arithmetic operator	Algebraic expression	Python expression
Addition	+	$f + 7$	<code>f + 7</code>
Subtraction	-	$p - c$	<code>p - c</code>
Multiplication	*	$b \cdot m$	<code>b * m</code>
Exponentiation	**	x	<code>x ** y</code>
True division	/	x/y or $\frac{x}{y}$ or $x \div y$	<code>x / y</code>

Floor division	//	$\lfloor x/y \rfloor$ or $\left\lfloor \frac{x}{y} \right\rfloor$ or	x // y
		$\lfloor x \div y \rfloor$	
Remainder (modulo)	%	r mod s	r % s

Multiplication (*)

Python uses the **asterisk (*) multiplication operator**:

```
In [1]: 7 * 4
Out[1]: 28
```

Exponentiation (**)

The **exponentiation (**) operator** raises one value to the power of another:

```
In [2]: 2 ** 10
Out[2]: 1024
```

To calculate the square root, you can use the exponent 1/2 (that is, 0.5):

```
In [3]: 9 ** (1 / 2)
Out[3]: 3.0
```

True Division (/) vs. Floor Division (//)

True division (/) divides a numerator by a denominator and yields a floating-point number with a decimal point, as in:

```
In [4]: 7 / 4
Out[4]: 1.75
```

Floor division (`//`) divides a numerator by a denominator, yielding the highest *integer* that's not greater than the result. Python **truncates** (discards) the fractional part:

```
In [5]: 7 // 4
```

```
Out[5]: 1
```

```
In [6]: 3 // 5
```

```
Out[6]: 0
```

```
In [7]: 14 // 7
```

```
Out[7]: 2
```

In true division, -13 divided by 4 gives -3.25 :

```
In [8]: -13 / 4
```

```
Out[8]: -3.25
```

Floor division gives the closest integer that's *not greater than* -3.25 —which is -4 :

```
In [9]: -13 // 4
```

```
Out[9]: -4
```

Exceptions and Tracebacks

Dividing by zero with `/` or `//` is not allowed and results in an **exception**—a sign that a problem occurred:

[lick here to view code image](#)

```
In [10]: 123 / 0
```

```
ZeroDivisionError
```

```
Traceback (most recent call last)
```

```
ipython-input-10-cd759d3fcf39> in <module>()
```

```
----> 1 123 / 0
```

```
ZeroDivisionError: division by zero
```

Python reports an exception with a **traceback**. This traceback indicates that an exception of type `ZeroDivisionError` occurred—most exception names end with `Error`. In interactive mode, the snippet number that caused the exception is specified

by the 10 in the line

```
<ipython-input-10-cd759d3fcf39> in <module>()
```

The line that begins with `---->` shows the code that caused the exception. Sometimes snippets have more than one line of code—the 1 to the right of `---->` indicates that line 1 within the snippet caused the exception. The last line shows the exception that occurred, followed by a colon (`:`) and an error message with more information about the exception:

```
ZeroDivisionError: division by zero
```

The “Files and Exceptions” chapter discusses exceptions in detail.

An exception also occurs if you try to use a variable that you have not yet created. The following snippet tries to add 7 to the undefined variable `z`, resulting in a `NameError`:

[lick here to view code image](#)

```
In [11]: z + 7
-----
NameError                                Traceback (most recent call last)
ipython-input-11-f2cdbf4fe75d> in <module>()
----> 1 z + 7

NameError: name 'z' is not defined
```

Remainder Operator

Python’s **remainder operator (%)** yields the remainder after the left operand is divided by the right operand:

```
In [12]: 17 % 5
Out[12]: 2
```

In this case, 17 divided by 5 yields a quotient of 3 and a remainder of 2. This operator is most commonly used with integers, but also can be used with other numeric types:

```
In [13]: 7.5 % 3.5
Out[13]: 0.5
```

Straight-Line Form

Algebraic notations such as

$$\frac{a}{b}$$

generally are not acceptable to compilers or interpreters. For this reason, algebraic expressions must be typed in **straight-line form** using Python's operators. The expression above must be written as `a / b` (or `a // b` for floor division) so that all operators and operands appear in a horizontal straight line.

Grouping Expressions with Parentheses

Parentheses group Python expressions, as they do in algebraic expressions. For example, the following code multiplies 10 times the quantity $5 + 3$:

```
In [14]: 10 * (5 + 3)
Out[14]: 80
```

Without these parentheses, the result is *different*:

```
In [15]: 10 * 5 + 3
Out[15]: 53
```

The parentheses are **redundant** (unnecessary) if removing them yields the *same* result.

Operator Precedence Rules

Python applies the operators in arithmetic expressions according to the following **rules of operator precedence**. These are generally the same as those in algebra:

1. Expressions in parentheses evaluate first, so parentheses may force the order of evaluation to occur in any sequence you desire. Parentheses have the highest level of precedence. In expressions with **nested parentheses**, such as $(a / (b - c))$, the expression in the *innermost* parentheses (that is, $b - c$) evaluates first.
2. Exponentiation operations evaluate next. If an expression contains several exponentiation operations, Python applies them from right to left.
3. Multiplication, division and modulus operations evaluate next. If an expression

contains several multiplication, true-division, floor-division and modulus operations, Python applies them from left to right. Multiplication, division and modulus are “on the same level of precedence.”

4. Addition and subtraction operations evaluate last. If an expression contains several addition and subtraction operations, Python applies them from left to right. Addition and subtraction also have the same level of precedence.

For the complete list of operators and their precedence (in lowest-to-highest order), see

<https://docs.python.org/3/reference/expressions.html#operator-precedence>

Operator Grouping

When we say that Python applies certain operators from left to right, we are referring to the operators’ **grouping**. For example, in the expression

```
a + b + c
```

the addition operators (+) group from left to right as if we parenthesized the expression as (a + b) + c. All Python operators of the same precedence group left-to-right except for the exponentiation operator (**), which groups right-to-left.

Redundant Parentheses

You can use redundant parentheses to group subexpressions to make the expression clearer. For example, the second-degree polynomial

```
y = a * x ** 2 + b * x + c
```

can be parenthesized, for clarity, as

[lick here to view code image](#)

```
y = (a * (x ** 2)) + (b * x) + c
```

Breaking a complex expression into a sequence of statements with shorter, simpler expressions also can promote clarity.

Operand Types

Each arithmetic operator may be used with integers and floating-point numbers. If both operands are integers, the result is an integer—except for the true-division (/) operator, which always yields a floating-point number. If both operands are floating-point numbers, the result is a floating-point number. Expressions containing an integer and a floating-point number are **mixed-type expressions**—these always produce floating-point results.

2.4 FUNCTION PRINT AND AN INTRO TO SINGLE- AND DOUBLE-QUOTED STRINGS

The built-in **print function** displays its argument(s) as a line of text:

[lick here to view code image](#)

```
In [1]: print('Welcome to Python!')
Welcome to Python!
```

In this case, the argument 'Welcome to Python!' is a string—a sequence of characters enclosed in single quotes ('). Unlike when you evaluate expressions in interactive mode, the text that `print` displays here is not preceded by `Out[1]`. Also, `print` does not display a string's quotes, though we'll soon show how to display quotes in strings.

You also may enclose a string in double quotes ("), as in:

[lick here to view code image](#)

```
In [2]: print("Welcome to Python!")
Welcome to Python!
```

Python programmers generally prefer single quotes. When `print` completes its task, it positions the screen cursor at the beginning of the next line.

Printing a Comma-Separated List of Items

The `print` function can receive a comma-separated list of arguments, as in:

[lick here to view code image](#)

```
In [3]: print('Welcome', 'to', 'Python!')
Welcome to Python!
```

t displays each argument separated from the next by a space, producing the same output as in the two preceding snippets. Here we showed a comma-separated list of strings, but the values can be of any type. We'll show in the next chapter how to prevent automatic spacing between values or use a different separator than space.

Printing Many Lines of Text with One Statement

When a backslash (\) appears in a string, it's known as the **escape character**. The backslash and the character immediately following it form an **escape sequence**. For example, \n represents the **newline character** escape sequence, which tells `print` to move the output cursor to the next line. The following snippet uses three newline characters to create several lines of output:

[lick here to view code image](#)

```
In [4]: print('Welcome\nto\n\nPython!')
Welcome
to

Python!
```

Other Escape Sequences

The following table shows some common escape sequences.

Escape sequence	Description
\n	Insert a newline character in a string. When the string is displayed, for each newline, move the screen cursor to the beginning of the next line.
\t	Insert a horizontal tab. When the string is displayed, for each tab, move the screen cursor to the next tab stop.
\\	Insert a backslash character in a string.

`\"` Insert a double quote character in a string.

`\'` Insert a single quote character in a string.

Ignoring a Line Break in a Long String

You may also split a long string (or a long statement) over several lines by using the **** **continuation character** as the last character on a line to ignore the line break:

[lick here to view code image](#)

```
In [5]: print('this is a longer string, so we \
...: split it over two lines')
this is a longer string, so we split it over two lines
```

The interpreter reassembles the string's parts into a single string with no line break. Though the backslash character in the preceding snippet is inside a string, it's not the escape character because another character does not follow it.

Printing the Value of an Expression

Calculations can be performed in `print` statements:

[lick here to view code image](#)

```
In [6]: print('Sum is', 7 + 3)
Sum is 10
```

2.5 TRIPLE-QUOTED STRINGS

Earlier, we introduced strings delimited by a pair of single quotes (') or a pair of double quotes ("). **Triple-quoted strings** begin and end with three double quotes ("""") or three single quotes ('''). The *Style Guide for Python Code* recommends three double quotes ("""). Use these to create:

- multiline strings,
- strings containing single or double quotes and
- **docstrings**, which are the recommended way to document the purposes of certain program components.

Including Quotes in Strings

In a string delimited by single quotes, you may include double-quote characters:

[lick here to view code image](#)

```
In [1]: print('Display "hi" in quotes')
Display "hi" in quotes
```

but not single quotes:

[lick here to view code image](#)

```
In [2]: print('Display 'hi' in quotes')
File "<ipython-input-2-19bf596ccf72>", line 1
    print('Display 'hi' in quotes')
                  ^
SyntaxError: invalid syntax
```

unless you use the `\'` escape sequence:

[lick here to view code image](#)

```
In [3]: print('Display \'hi\' in quotes')
Display 'hi' in quotes
```

Snippet [2] displayed a syntax error due to a single quote inside a single-quoted string. IPython displays information about the line of code that caused the syntax error and points to the error with a `^` symbol. It also displays the message `SyntaxError: invalid syntax`.

A string delimited by double quotes may include single quote characters:

[lick here to view code image](#)

```
In [4]: print("Display the name O'Brien")
Display the name O'Brien
```

but not double quotes, unless you use the `\` escape sequence:

[lick here to view code image](#)

```
In [5]: print("Display \"hi\" in quotes")
Display "hi" in quotes
```

To avoid using `\` ' and `\` " inside strings, you can enclose such strings in triple quotes:

[lick here to view code image](#)

```
In [6]: print("""Display "hi" and 'bye' in quotes""")
Display "hi" and 'bye' in quotes
```

Multiline Strings

The following snippet assigns a multiline triple-quoted string to `triple_quoted_string`:

[lick here to view code image](#)

```
In [7]: triple_quoted_string = """This is a triple-quoted
...: string that spans two lines"""
```

IPython knows that the string is incomplete because we did not type the closing `"""` before we pressed *Enter*. So, IPython displays a **continuation prompt** `...:` at which you can input the multiline string's next line. This continues until you enter the ending `"""` and press *Enter*. The following displays `triple_quoted_string`:

[lick here to view code image](#)

```
In [8]: print(triple_quoted_string)
This is a triple-quoted
string that spans two lines
```

Python stores multiline strings with embedded newline characters. When we evaluate `triple_quoted_string` rather than printing it, IPython displays it in single quotes

with a `\n` character where you pressed *Enter* in snippet [7]. The quotes IPython displays indicate that `triple_quoted_string` is a string—they're not part of the string's contents:

[lick here to view code image](#)

```
In [9]: triple_quoted_string
Out[9]: 'This is a triple-quoted\nstring that spans two    lines'
```

2.6 GETTING INPUT FROM THE USER

The built-in **`input`** function requests and obtains user input:

[lick here to view code image](#)

```
In [1]: name = input("What's    your name? ")
What's your name? Paul

In [2]: name
Out[2]: 'Paul'

In [3]: print(name)
Paul
```

The snippet executes as follows:

- First, `input` displays its string argument—a prompt—to tell the user what to type and waits for the user to respond. We typed `Paul` and pressed *Enter*. We use **bold** text to distinguish the user's input from the prompt text that `input` displays.
- Function `input` then returns those characters as a string that the program can use. Here we assigned that string to the variable `name`.

Snippet [2] shows `name`'s value. Evaluating `name` displays its value in single quotes as `'Paul'` because it's a string. Printing `name` (in snippet [3]) displays the string without the quotes. If you enter quotes, they're part of the string, as in:

[lick here to view code image](#)

```
In [4]: name = input("What's    your name? ")
What's your name? 'Paul'
```

```
In [5]: name
Out[5]: "'Paul'"

In [6]: print(name)
'Paul'
```

Function `input` Always Returns a String

Consider the following snippets that attempt to read two numbers and add them:

[lick here to view code image](#)

```
In [7]: value1 = input('Enter first   number: ')
Enter first number: 7

In [8]: value2 = input('Enter second   number: ')
Enter second number: 3

In [9]: value1 + value2
Out[9]: '73'
```

Rather than adding the integers 7 and 3 to produce 10, Python “adds” the *string* values '7' and '3', producing the *string* '73'. This is known as **string concatenation**. It creates a new string containing the left operand’s value followed by the right operand’s value.

Getting an Integer from the User

If you need an integer, convert the string to an integer using the built-in **`int` function**:

[lick here to view code image](#)

```
In [10]: value = input('Enter an   integer: ')
Enter an integer: 7

In [11]: value = int(value)

In [12]: value
Out[12]: 7
```

We could have combined the code in snippets [10] and [11]:

[lick here to view code image](#)

```
In [13]: another_value = int(input('Enter another integer: '))
Enter another integer: 13

In [14]: another_value
Out[14]: 13
```

Variables `value` and `another_value` now contain integers. Adding them produces an integer result (rather than concatenating them):

```
In [15]: value + another_value
Out[15]: 20
```

If the string passed to `int` cannot be converted to an integer, a `ValueError` occurs:

[lick here to view code image](#)

```
In [16]: bad_value = int(input('Enter another integer: '))
Enter another integer: hello
-----
ValueError                                Traceback (most recent call last)
ipython-input-16-cd36e6cf8911> in <module>()
----> 1 bad_value = int(input('Enter another integer: '))

ValueError: invalid literal for int() with base 10: 'hello'
```

Function `int` also can convert a floating-point value to an integer:

```
In [17]: int(10.5)
Out[17]: 10
```

To convert strings to floating-point numbers, use the built-in **`float` function**.

2.7 DECISION MAKING: THE `IF` STATEMENT AND COMPARISON OPERATORS

A **condition** is a Boolean expression with the value **`True`** or **`False`**. The following determines whether 7 is greater than 4 and whether 7 is less than 4:

```
In [1]: 7 > 4
Out[1]: True
```

```
In [2]: 7 < 4
Out[2]: False
```

True and False are Python keywords. Using a keyword as an identifier causes a Syntax-Error. True and False are each capitalized.

You'll often create conditions using the **comparison operators** in the following table:

Algebraic operator	Python operator	Sample condition	Meaning
>	>	<code>x > y</code>	x is greater than y
<	<	<code>x < y</code>	x is less than y
≥	>=	<code>x >= y</code>	x is greater than or equal to y
≤	<=	<code>x <= y</code>	x is less than or equal to y
=	==	<code>x == y</code>	x is equal to y
≠	!=	<code>x != y</code>	x is not equal to y

Operators >, <, >= and <= all have the same precedence. Operators == and != both have the same precedence, which is lower than that of >, <, >= and <=. A syntax error occurs when any of the operators ==, !=, >= and <= contains spaces between its pair of symbols:

[lick here to view code image](#)

```
In [3]: 7 > = 4
File "<ipython-input-3-5c6e2897f3b3>", line 1
      7 > = 4
        ^
SyntaxError: invalid syntax
```

Another syntax error occurs if you reverse the symbols in the operators `!=`, `>=` and `<=` (by writing them as `=!`, `=>` and `=<`).

Making Decisions with the `if` Statement: Introducing Scripts

We now present a simple version of the **`if` statement**, which uses a condition to decide whether to execute a statement (or a group of statements). Here we'll read two integers from the user and compare them using six consecutive `if` statements, one for each comparison operator. If the condition in a given `if` statement is `True`, the corresponding `print` statement executes; otherwise, it's skipped.

IPython interactive mode is helpful for executing brief code snippets and seeing immediate results. When you have many statements to execute as a group, you typically write them as a **script** stored in a file with the `.py` (short for Python) extension—such as `fig02_01.py` for this example's script. Scripts are also called programs. For instructions on locating and executing the scripts in this book, see [chapter 1's IPython Test-Drive](#).

Each time you execute this script, three of the six conditions are `True`. To show this, we execute the script three times—once with the first integer *less than* the second, once with the *same* value for both integers and once with the first integer *greater than* the second. The three sample executions appear after the script

Each time we present a script like the one below, we introduce it before the figure, then explain the script's code after the figure. We show line numbers for your convenience—these are not part of Python. IDEs enable you to choose whether to display line numbers. To run this example, change to this chapter's `ch02 examples` folder, then enter:

```
ipython fig02_01.py
```

or, if you're in IPython already, you can use the command:

run fig02_01.py

[lick here to view code image](#)

```
1 # fig02_01.py
2 """Comparing integers using if statements and comparison operators."""
3
4 print('Enter two integers, and I will tell you',
5       'the relationships they satisfy.')
6
7 # read first integer
8 number1 = int(input('Enter first integer: '))
9
10 # read second integer
11 number2 = int(input('Enter second integer: '))
12
13 if number1 == number2:
14     print(number1, 'is equal to', number2)
15
16 if number1 != number2:
17     print(number1, 'is not equal to', number2)
18
19 if number1 < number2:
20     print(number1, 'is less than', number2)
21
22 if number1 > number2:
23     print(number1, 'is greater than', number2)
24
25 if number1 <= number2:
26     print(number1, 'is less than or equal to', number2)
27
28 if number1 >= number2:
29     print(number1, 'is greater than or equal to', number2)
```

[lick here to view code image](#)

```
Enter two integers and I will tell you the relationships they satisfy.
Enter first integer: 37
Enter second integer: 42
37 is not equal to 42
37 is less than 42
37 is less than or equal to 42
```

[lick here to view code image](#)

```
Enter two integers and I will tell you the relationships they satisfy.  
Enter first integer: 7  
Enter second integer: 7  
7 is equal to 7  
7 is less than or equal to 7  
7 is greater than or equal to 7
```

[lick here to view code image](#)

```
Enter two integers and I will tell you the relationships they satisfy.  
Enter first integer: 54  
Enter second integer: 17  
54 is not equal to 17  
54 is greater than 17  
54 is greater than or equal to 17
```

Comments

Line 1 begins with the hash character (**#**), which indicates that the rest of the line is a **comment**:

```
# fig02_01.py
```

For easy reference, we begin each script with a comment indicating the script's file name. A comment also can begin to the right of the code on a given line and continue until the end of that line.

Docstrings

The *Style Guide for Python Code* states that each script should start with a docstring that explains the script's purpose, such as the one in line 2:

```
"""Comparing integers using if statements and comparison operators."""
```

For more complex scripts, the docstring often spans many lines. In later chapters, you'll use docstrings to describe script components you define, such as new functions and new types called classes. We'll also discuss how to access docstrings with the IPython help mechanism.

Blank Lines

Line 3 is a blank line. You use blank lines and space characters to make code easier to read. Together, blank lines, space characters and tab characters are known as **white space**. Python ignores most white space—you'll see that some indentation is required.

Splitting a Lengthy Statement Across Lines

Lines 4–5

[lick here to view code image](#)

```
print('Enter two integers, and I will tell you',  
      'the relationships they satisfy.')
```

display instructions to the user. These are too long to fit on one line, so we broke them into two strings. Recall that you can display several values by passing to `print` a comma-separated list—`print` separates each value from the next with a space.

Typically, you write statements on one line. You may spread a lengthy statement over several lines with the `\` continuation character. Python also allows you to split long code lines in parentheses without using continuation characters (as in lines 4–5). This is the preferred way to break long code lines according to the *Style Guide for Python Code*. Always choose breaking points that make sense, such as after a comma in the preceding call to `print` or before an operator in a lengthy expression.

Reading Integer Values from the User

Next, lines 8 and 11 use the built-in `input` and `int` functions to prompt for and read two integer values from the user.

`if` Statements

The `if` statement in lines 13–14

[lick here to view code image](#)

```
if number1 == number2:  
    print(number1, 'is equal to', number2)
```

uses the `==` comparison operator to determine whether the values of variables `number1` and `number2` are equal. If so, the condition is `True`, and line 14 displays a

line of text indicating that the values are equal. If any of the remaining `if` statements' conditions are `True` (lines 16, 19, 22, 25 and 28), the corresponding `print` displays a line of text.

Each `if` statement consists of the keyword `if`, the condition to test, and a colon (`:`) followed by an indented body called a **suite**. Each suite must contain one or more statements. Forgetting the colon (`:`) after the condition is a common syntax error.

Suite Indentation

Python requires you to indent the statements in suites. The *Style Guide for Python Code* recommends four-space indents—we use that convention throughout this book. You'll see in the next chapter that incorrect indentation can cause errors.

Confusing `==` and `=`

Using the assignment symbol (`=`) instead of the equality operator (`==`) in an `if` statement's condition is a common syntax error. To help avoid this, read `==` as “is equal to” and `=` as “is assigned.” You'll see in the next chapter that using `==` in place of `=` in an assignment statement can lead to subtle problems.

Chaining Comparisons

You can chain comparisons to check whether a value is in a range. The following comparison determines whether `x` is in the range 1 through 5, inclusive:

```
In [1]: x = 3

In [2]: 1 <= x <= 5
Out[2]: True

In [3]: x = 10

In [4]: 1 <= x <= 5
Out[4]: False
```

Precedence of the Operators We've Presented So Far

The precedence of the operators introduced in this chapter is shown below:

Operators	Grouping	Type
-----------	----------	------

()	left to right	parentheses
**	right to left	exponentiation
* / // %	left to right	multiplication, true division, floor division, remainder
+ -	left to right	addition, subtraction
> <= < >=	left to right	less than, less than or equal, greater than, greater than or equal
== !=	left to right	equal, not equal

The table lists the operators top-to-bottom in decreasing order of precedence. When writing expressions containing multiple operators, confirm that they evaluate in the order you expect by referring to the operator precedence chart at

<https://docs.python.org/3/reference/expressions.html#operator-precedence>

2.8 OBJECTS AND DYNAMIC TYPING

Values such as 7 (an integer), 4.1 (a floating-point number) and 'dog' are all objects. Every object has a type and a value:

```
In [1]: type(7)
Out[1]: int

In [2]: type(4.1)
```

```
Out[2]: float

In [3]: type('dog')
Out[3]: str
```

An object's value is the data stored in the object. The snippets above show objects of built-in types **int** (for integers), **float** (for floating-point numbers) and **str** (for strings).

Variables Refer to Objects

Assigning an object to a variable **binds** (associates) that variable's name to the object. As you've seen, you can then use the variable in your code to access the object's value:

```
In [4]: x = 7

In [5]: x + 10
Out[5]: 17

In [6]: x
Out[6]: 7
```

After snippet [4]'s assignment, the variable `x` **refers** to the integer object containing 7. As shown in snippet [6], snippet [5] does not change `x`'s value. You can change `x` as follows:

```
In [7]: x = x + 10

In [8]: x
Out[8]: 17
```

Dynamic Typing

Python uses **dynamic typing**—it determines the type of the object a variable refers to while executing your code. We can show this by rebinding the variable `x` to different objects and checking their types:

```
In [9]: type(x)
Out[9]: int

In [10]: x = 4.1

In [11]: type(x)
Out[11]: float
```

```
In [12]: x = 'dog'
```

```
In [13]: type(x)  
Out[13]: str
```

Garbage Collection

Python creates objects in memory and removes them from memory as necessary. After snippet [10], the variable `x` now refers to a `float` object. The integer object from snippet [7] is no longer bound to a variable. As we'll discuss in a later chapter, Python automatically removes such objects from memory. This process—called **garbage collection**—helps ensure that memory is available for new objects you create.

2.9 INTRO TO DATA SCIENCE: BASIC DESCRIPTIVE STATISTICS

In data science, you'll often use statistics to describe and summarize your data. Here, we begin by introducing several such **descriptive statistics**, including:

- **minimum**—the smallest value in a collection of values.
- **maximum**—the largest value in a collection of values.
- **range**—the range of values from the minimum to the maximum.
- **count**—the number of values in a collection.
- **sum**—the total of the values in a collection.

We'll look at determining the *count* and *sum* in the next chapter. **Measures of dispersion** (also called **measures of variability**), such as *range*, help determine how spread out values are. Other measures of dispersion that we'll present in later chapters include *variance* and *standard deviation*.

Determining the Minimum of Three Values

First, let's show how to determine the minimum of three values manually. The following script prompts for and inputs three values, uses `if` statements to determine the minimum value, then displays it.

[lick here to view code image](#)

```

1 # fig02_02.py
2 """Find the minimum of three values."""
3
4 number1 = int(input('Enter first integer: '))
5 number2 = int(input('Enter second integer: '))
6 number3 = int(input('Enter third integer: '))
7
8 minimum = number1
9
10 if number2 < minimum:
11     minimum = number2
12
13 if number3 < minimum:
14     minimum = number3
15
16 print('Minimum value is', minimum)

```

[lick here to view code image](#)

```

Enter first integer: 12
Enter second integer: 27
Enter third integer: 36
Minimum value is 12

```

[lick here to view code image](#)

```

Enter first integer: 27
Enter second integer: 12
Enter third integer: 36
Minimum value is 12

```

[lick here to view code image](#)

```

Enter first integer: 36
Enter second integer: 27
Enter third integer: 12
Minimum value is 12

```

After inputting the three values, we process one value at a time:

- First, we assume that `number1` contains the smallest value, so line 8 assigns it to the variable `minimum`. Of course, it's possible that `number2` or `number3` contains

the actual smallest value, so we still must compare each of these with `minimum`.

- The first `if` statement (lines 10–11) then tests `number2 < minimum` and if this condition is `True` assigns `number2` to `minimum`.
- The second `if` statement (lines 13–14) then tests `number3 < minimum`, and if this condition is `True` assigns `number3` to `minimum`.

Now, `minimum` contains the smallest value, so we display it. We executed the script three times to show that it always finds the smallest value regardless of whether the user enters it first, second or third.

Determining the Minimum and Maximum with Built-In Functions `min` and `max`

Python has many built-in functions for performing common tasks. Built-in functions `min` and `max` calculate the minimum and maximum, respectively, of a collection of values:

```
In [1]: min(36, 27, 12)
Out[1]: 12

In [2]: max(36, 27, 12)
Out[2]: 36
```

The functions `min` and `max` can receive any number of arguments.

Determining the Range of a Collection of Values

The *range* of values is simply the minimum through the maximum value. In this case, the range is 12 through 36. Much data science is devoted to getting to know your data. Descriptive statistics is a crucial part of that, but you also have to understand how to interpret the statistics. For example, if you have 100 numbers with a range of 12 through 36, those numbers could be distributed evenly over that range. At the opposite extreme, you could have clumping with 99 values of 12 and one 36, or one 12 and 99 values of 36.

Functional-Style Programming: Reduction

Throughout this book, we introduce various *functional-style programming* capabilities. These enable you to write code that can be more concise, clearer and easier to **debug**—that is, find and correct errors. The `min` and `max` functions are examples of

functional-style programming concept called **reduction**. They reduce a collection of values to a *single* value. Other reductions you'll see include the sum, average, variance and standard deviation of a collection of values. You'll also see how to define custom reductions.

Upcoming Intro to Data Science Sections

In the next two chapters, we'll continue our discussion of basic descriptive statistics with *measures of central tendency*, including *mean*, *median* and *mode*, and *measures of dispersion*, including *variance* and *standard deviation*.

2.10 WRAP-UP

This chapter continued our discussion of arithmetic. You used variables to store values for later use. We introduced Python's arithmetic operators and showed that you must write all expressions in straight-line form. You used the built-in function `print` to display data. We created single-, double- and triple-quoted strings. You used triple-quoted strings to create multiline strings and to embed single or double quotes in strings.

You used the `input` function to prompt for and get input from the user at the keyboard. We used the functions `int` and `float` to convert strings to numeric values. We presented Python's comparison operators. Then, you used them in a script that read two integers from the user and compared their values using a series of `if` statements.

We discussed Python's dynamic typing and used the built-in function `type` to display an object's type. Finally, we introduced the basic descriptive statistics `minimum` and `maximum` and used them to calculate the range of a collection of values. In the next chapter, we present Python's control statements.

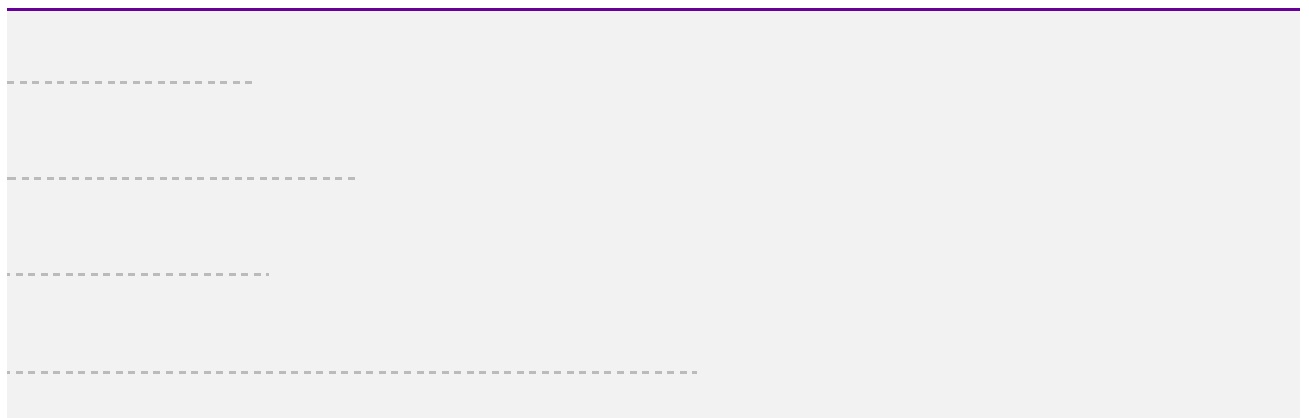
3. Control Statements

Objectives

In this chapter, you'll:

- Make decisions with `if`, `if else` and `if elif else`.
- Execute statements repeatedly with `while` and `for`.
- Shorten assignment expressions with augmented assignments.
- Use the `for` statement and the built-in `range` function to repeat actions for a sequence of values.
- Perform sentinel-controlled iteration with `while`.
- Create compound conditions with the Boolean operators `and`, `or` and `not`.
- Stop looping with `break`.
- Force the next iteration of a loop with `continue`.
- Use functional-style programming features to write scripts that are more concise, clearer, easier to debug and easier to parallelize.

Outline



.5 `while` Statement

.6 `for` Statement

.6.1 Iterables, Lists and Iterators

.6.2 Built-In `range` **Function**

.7 Augmented Assignments

.8 Sequence-Controlled Iteration; Formatted Strings

.9 Sentinel-Controlled Iteration

.10 Built-In Function `range`: A Deeper Look

.11 Using Type `Decimal` for Monetary Amounts

.12 `break` and `continue` Statements

.13 Boolean Operators `and`, `or` and `not`

.14 Intro to Data Science: Measures of Central Tendency—Mean, Median and Mode

.15 Wrap-Up

3.1 INTRODUCTION

In this chapter, we present Python’s control statements—`if`, `if else`, `if elif else`, `while`, `for`, `break` and `continue`. You’ll use the `for` statement to perform sequence-controlled- iteration—you’ll see that the number of items in a sequence of item determines the `for` statement’s number of iterations. You’ll use the built-in function `range` to generate sequences of integers.

We’ll show sentinel-controlled iteration with the `while` statement. You’ll use the Python Standard Library’s `Decimal` type for precise monetary calculations. We’ll format data in f-strings (that is, format strings) using various format specifiers. We’ll also show the Boolean operators `and`, `or` and `not` for creating compound conditions. In the Intro to Data Science section, we’ll consider measures of central tendency—mean, median and mode—using the Python Standard Library’s `statistics` module.

.2 CONTROL STATEMENTS

Python provides three selection statements that execute code based on a condition that evaluates to either `True` or `False`:

- The `if` statement performs an action if a condition is `True` or skips the action if the condition is `False`.
- The **`if...else` statement** performs an action if a condition is `True` or performs a *different* action if the condition is `False`.
- The **`if...elif...else` statement** performs one of many different actions, depending on the truth or falsity of *several* conditions.

Anywhere a single action can be placed, a group of actions can be placed.

Python provides two **iteration statements**—`while` and `for`:

- The **`while` statement** repeats an action (or a group of actions) as long as a condition remains `True`.
- The **`for` statement** repeats an action (or a group of actions) for every item in a sequence of items.

Keywords

The words `if`, `elif`, `else`, `while`, `for`, `True` and `False` are Python keywords. Using a keyword as an identifier such as a variable name is a syntax error. The following table lists Python's keywords.

Python keywords				
<code>and</code>	<code>as</code>	<code>assert</code>	<code>async</code>	<code>await</code>
<code>break</code>	<code>class</code>	<code>continue</code>	<code>def</code>	<code>del</code>

elif	else	except	False	finally
for	from	global	if	import
in	is	lambda	None	nonlocal
not	or	pass	raise	return
True	try	while	with	yield

3.3 IF STATEMENT

Let's execute a Python `if` statement:

[lick here to view code image](#)

```
In [1]: grade = 85
In [2]: if grade >= 60:
...:     print('Passed')
...:
Passed
```

The condition `grade >= 60` is `True`, so the indented `print` statement in the `if`'s suite displays `'Passed'`.

Suite Indentation

Indenting a suite is required; otherwise, an `IndentationError` syntax error occurs:

[lick here to view code image](#)

```
In [3]: if grade >= 60:
...: print('Passed') # statement is not indented properly
File "<ipython-input-3-f42783904220>", line 2
    print('Passed') # statement is not indented properly
```

```
^
IndentationError: expected an indented block
```

An `IndentationError` also occurs if you have more than one statement in a suite and those statements do not have the *same* indentation:

[lick here to view code image](#)

```
In [4]: if grade >= 60:
...:     print('Passed') # indented 4 spaces
...:     print('Good job!') # incorrectly indented only two spaces
File <ipython-input-4-8c0d75c127bf>, line 3
    print('Good job!') # incorrectly indented only two spaces
    ^
IndentationError: unindent does not match any outer indentation level
```

Sometimes error messages may not be clear. The fact that Python calls attention to the line is usually enough for you to figure out what's wrong. Apply indentation conventions uniformly throughout your code—programs that are not uniformly indented are hard to read.

Every Expression Can Be Interpreted as Either `True` or `False`

You can base decisions on *any* expression. A nonzero value is `True`. Zero is `False`:

[lick here to view code image](#)

```
In [5]: if 1:
...:     print('Nonzero values are true, so this will print')
...:
Nonzero values are true, so this will print

In [6]: if 0:
...:     print('Zero is false, so this will not print')

In [7]:
```

Strings containing characters are `True` and empty strings (`' '`, `""` or `"""`) are `False`.

Confusing `==` and `=`

Using the equality operator `==` instead of `=` in an assignment statement can lead to subtle problems. For example, in this session, snippet [1] defined `grade` with the

assignment:

```
grade = 85
```

If instead we accidentally wrote:

```
grade == 85
```

then `grade` would be undefined and we'd get a `NameError`. If `grade` had been defined before the preceding statement, then `grade == 85` would simply evaluate to `True` or `False`, and not perform an assignment. This is a logic error.

3.4 IF ELSE AND IF ELIF ELSE STATEMENTS

The `if else` statement executes different suites, based on whether a condition is `True` or `False`:

[lick here to view code image](#)

```
In [1]: grade = 85

In [2]: if grade >= 60:
...:     print('Passed')
...: else:
...:     print('Failed')
...:
Passed
```

The condition above is `True`, so the `if` suite displays `'Passed'`. Note that when you press *Enter* after typing `print('Passed')`, IPython indents the next line four spaces. You must delete those four spaces so that the `else:` suite correctly aligns under the `i` in `if`.

The following code assigns 57 to the variable `grade`, then shows the `if else` statement again to demonstrate that only the `else` suite executes when the condition is `False`:

[lick here to view code image](#)

```
In [3]: grade = 57
```

```
In [4]: if grade >= 60:
...:     print('Passed')
...: else:
...:     print('Failed')
...:
Failed
```

Use the up and down arrow keys to navigate backwards and forwards through the current interactive session's snippets. Pressing *Enter* re-executes the snippet that's displayed. Let's set `grade` to 99, press the up arrow key twice to recall the code from snippet [4], then press *Enter* to re-execute that code as snippet [6]. Every recalled snippet that you execute gets a new ID:

[lick here to view code image](#)

```
In [5]: grade = 99

In [6]: if grade >= 60:
...:     print('Passed')
...: else:
...:     print('Failed')
...:
Passed
```

Conditional Expressions

Sometimes the suites in an `if else` statement assign different values to a variable, based on a condition, as in:

[lick here to view code image](#)

```
In [7]: grade = 87

In [8]: if grade >= 60:
...:     result = 'Passed'
...: else:
...:     result = 'Failed'
...:
```

We can then print or evaluate that variable:

```
In [9]: result
Out[9]: 'Passed'
```

You can write statements like snippet [8] using a concise **conditional expression**:

[lick here to view code image](#)

```
In [10]: result = ('Passed' if grade >= 60 else 'Failed')

In [11]: result
Out[11]: 'Passed'
```

The parentheses are not required, but they make it clear that the statement assigns the conditional expression's value to `result`. First, Python evaluates the condition `grade >= 60`:

- If it's `True`, snippet [10] assigns to `result` the value of the expression to the *left* of `if`, namely `'Passed'`. The `else` part does not execute.
- If it's `False`, snippet [10] assigns to `result` the value of the expression to the *right* of `else`, namely `'Failed'`.

In interactive mode, you also can evaluate the conditional expression directly, as in:

[lick here to view code image](#)

```
In [12]: 'Passed' if grade >= 60 else 'Failed'
Out[12]: 'Passed'
```

Multiple Statements in a Suite

The following code shows two statements in the `else` suite of an `if...else` statement:

[lick here to view code image](#)

```
In [13]: grade = 49

In [14]: if grade >= 60:
...:     print('Passed')
...: else:
...:     print('Failed')
...:     print('You must take    this course again')
...:
Failed
You must take this course again
```